# ConfigMap

Цели работы:

- изучить концепцию ConfigMap и ее назначение в Kubernetes.
- освоить способы использования ConfigMap: как переменные окружения, как файлы конфигурации через Volumes.
- научиться создавать и применять ConfigMap.
- понять разницу между явным указанием ключей (items) и монтированием всей ConfigMap.

`ConfigMap` — это объект Kubernetes, предназначенный для хранения нефункциональных данных в формате "ключ-значение". ConfigMap позволяет отделить конфигурацию приложения от его образа, что делает приложение более переносимым и гибким.

Проблемы, которые решает `ConfigMap`:

- разные конфигурации для разных сред (разработка, тестирование и т д),
- необходимость менять конфигурацию без пересборки Docker-образа,
- хранение конфигурационных файлов в структурированном виде.

Способы использования ConfigMap в Pod:

- как переменные окружения: вся ConfigMap или отдельные ключи могут быть подставлены в качестве переменных окружения контейнера,
- как аргументы командной строки: значения из ConfigMap можно использовать в качестве аргументов для команды запуска контейнера,
- как файлы в Volume: ConfigMap может быть смонтирована в контейнер в виде одного или нескольких файлов. Это позволяет работать с конфигурациями в формате .properties, .ini, .json и т.д.

**Пример 1. ConfigMap как переменные окружения**

Создание ConfigMap и использование ее данных для установки переменных окружения в Pod *(показывается базовый способ использования ConfigMap для динамической конфигурации).*

**Шаг 1.** Создаем ConfigMap.

01-configmap-env.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  DATABASE_HOST: "postgres-prod.example.com"
  DATABASE_PORT: "5432"
  LOG_LEVEL: "INFO"
```

```
C:\Temp>kubectl apply -f 01-configmap-env.yaml
configmap/app-config created

C:\Temp>kubectl get configmaps
NAME               DATA   AGE
app-config         3      8s
kube-root-ca.crt   1      26d

C:\Temp>kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
DATABASE_HOST:
----
postgres-prod.example.com

DATABASE_PORT:
----
5432

LOG_LEVEL:
----
INFO
```

**Шаг 2.** Создаем Pod, который использует эти данные.

01-pod-env-config.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh"]
    args: ["-c", "sleep 3600"]
    env:
    - name: DB_HOST
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DATABASE_HOST
    - name: DB_PORT
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DATABASE_PORT
```

```
C:\Temp>kubectl apply -f 01-configmap-env-pod.yaml
pod/configmap-env-pod created

C:\Temp>kubectl exec configmap-env-pod -- env | findstr "DB_"
DB_PORT=5432
DB_HOST=postgres-prod.example.com
```

***Примечание**: в Linux:* `kubectl exec configmap-env-pod -- env | grep DB_`

**Пример 2.** ConfigMap как файлы в Volume (Без items)

Монтирование всей ConfigMap в директорию контейнера. Каждый ключ становится файлом *(простой способ предоставить приложению доступ ко всем конфигурационным данным сразу).*

**Шаг 1.** Создаем ConfigMap с конфигурационными файлами.

02-configmap-files.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-file-config
data:
  application.properties: |
    server.port=8080
    spring.datasource.url=jdbc:postgresql://localhost/test
  config.ini: |
    [database]
    host=localhost
    port=5432
```

**Шаг 2.** Создаем Pod и монтируем ConfigMap как Volume.

02-pod-volume-config.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh"]
    args: ["-c", "sleep 3600"]
    volumeMounts:
    - name: config-volume
      mountPath: /etc/app-config
  volumes:
  - name: config-volume
    configMap:
      name: app-file-config
```

Проверим

```
C:\Temp>kubectl apply -f 02-pod-volume-config.yaml
pod/configmap-volume-pod created
```

```
C:\Temp>kubectl exec configmap-volume-pod -- ls -la /etc/app-config/
total 12
drwxrwxrwx    3 root     root          4096 Oct 22 19:16 .
drwxr-xr-x    1 root     root          4096 Oct 22 19:16 ..
drwxr-xr-x    2 root     root          4096 Oct 22 19:16 ..2025_10_22_19_16_04.1805952885
lrwxrwxrwx    1 root     root            32 Oct 22 19:16 ..data -> ..2025_10_22_19_16_04.1805952885
lrwxrwxrwx    1 root     root            29 Oct 22 19:16 application.properties -> ..data/application.properties
lrwxrwxrwx    1 root     root            17 Oct 22 19:16 config.ini -> ..data/config.ini

C:\Temp>kubectl exec configmap-volume-pod -- cat /etc/app-config/config.ini
[database]
host=localhost
port=5432

C:\Temp>kubectl exec configmap-volume-pod -- cat /etc/app-config/application.properties
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost/test
```

Вывод:

- ConfigMap app-file-config успешно смонтирован
- содержит 2 файла: application.properties и config.ini
- механизм автоматического обновления ConfigMap настроен правильно
- Pod имеет доступ к конфигурационным файлам

**Пример 3. ConfigMap как файлы в Volume (С items)**

Точное управление тем, какие ключи и под какими именами монтируются в контейнер *(используется для тонкой настройки, когда нужно переименовать файлы или смонтировать только часть ключей).*

**Шаг 1.** Используем ту же ConfigMap app-file-config.
**Шаг 2.** Создаем Pod с явным указанием items.

03-pod-items-config.yaml – Блокнот

Файл Правка Формат Вид Справка

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-items-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh"]
    args: ["-c", "sleep 3600"]
    volumeMounts:
    - name: config-volume
      mountPath: /etc/app-config
  volumes:
  - name: config-volume
    configMap:
      name: app-file-config
      items:
      - key: application.properties
        path: app.properties
      - key: config.ini
        path: settings.ini
```

```
C:\Temp>kubectl apply -f 03-pod-items-config.yaml
pod/configmap-volume-items-pod created

C:\Temp>kubectl exec configmap-volume-items-pod -- ls -la /etc/app-config/
total 12
drwxrwxrwx    3 root     root         4096 Oct 22 19:30 .
drwxr-xr-x    1 root     root         4096 Oct 22 19:30 ..
drwxr-xr-x    2 root     root         4096 Oct 22 19:30 ..2025_10_22_19_30_06.3675188251
lrwxrwxrwx    1 root     root           32 Oct 22 19:30 ..data -> ..2025_10_22_19_30_06.3675188251
lrwxrwxrwx    1 root     root           21 Oct 22 19:30 app.properties -> ..data/app.properties
lrwxrwxrwx    1 root     root           19 Oct 22 19:30 settings.ini -> ..data/settings.ini
```

**Сравнение подходов с items и без:**

- **Без** items: Все ключи монтируются как файлы с именами, соответствующими ключам. Простота, но меньше контроля.
- **С** items: Позволяет выбрать конкретные ключи и задать им новые имена в файловой системе контейнера. Больше контроля, подходит для сложных конфигураций.

**Пример 4.** Веб-приложение с динамическим контентом из ConfigMap

Демонстрация практического использования ConfigMap для управления контентом веб-приложения без пересборки образа. Показывает, как можно динамически менять содержимое веб-страницы через ConfigMap.

**Шаг 1.** Создаем ConfigMap с HTML-контентом
**Шаг 2.** Создаем Deployment с монтированием ConfigMap
**Шаг 3.** Создаем Service для внешнего доступа

*Все три шага можно описать в одном yaml-манифесте, разделяя их (---)*

ex-deploy-service-config.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  message: "Hello from ConfigMap!"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
      - name: nginx
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
```

ex-deploy-service-config.yaml – Блокнот

Файл Правка Формат Вид Справка

```yaml
      containers:
      - name: nginx
        image: nginx:alpine
        ports:
        - containerPort: 80
        volumeMounts:
        - name: html-vol
          mountPath: /usr/share/nginx/html
      volumes:
      - name: html-vol
        configMap:
          name: app-config
          items:
          - key: message
            path: index.html
---
apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
  - port: 80
    nodePort: 30080
```

**Шаг 4.** Проверяем работу

Применим конфигурацию:
Проверим созданные ресурсы:

```
C:\Temp>kubectl apply -f ex-deploy-service-config.yaml
configmap/app-config configured
deployment.apps/web-app created
service/web-service created

C:\Temp>kubectl get configmap app-config
NAME         DATA   AGE
app-config   1      68m

C:\Temp>kubectl get pods -l app=web-app
NAME                       READY   STATUS    RESTARTS   AGE
web-app-f998c9c77-m9wf5    1/1     Running   0          9m25s

C:\Temp>kubectl get service web-service
NAME          TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
web-service   NodePort   10.109.216.235   <none>        80:30080/TCP   9m31s
```

Тестируем в браузере:

**Шаг 5.** Динамическое обновление контента

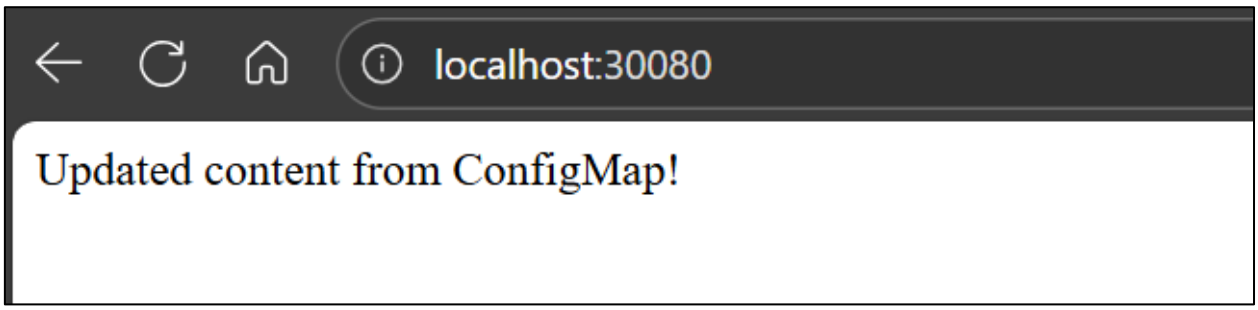
Обновим ConfigMap:

```
kubectl edit configmap app-config
```

Изменим значение:

```
data:
  message: "Updated content from ConfigMap!
```

Контент автоматически обновится в Pod'ах (может потребоваться несколько секунд).



**Задания для самостоятельного решения**

**Задание 7.1**. Базовое использование ConfigMap
Создайте ConfigMap web-config со следующими данными:
```
APP_COLOR: blue
APP_ENV: prod
```

Создайте Pod web-pod, который использует эти значения как переменные окружения COLOR и ENVIRONMENT соответственно.

*Проверочные команды:*
```
kubectl get configmap web-config -o yaml
kubectl exec web-pod -- env | grep -E '(COLOR|ENVIRONMENT)'
```

**Задание 7.2.** ConfigMap как файлы
Создайте ConfigMap nginx-config с ключом default.conf, значением которого будет базовая конфигурация nginx.

Создайте Pod nginx-pod с контейнером nginx, который монтирует эту ConfigMap в директорию /etc/nginx/conf.d/.

*Проверочные команды:*
```
kubectl exec nginx-pod -- cat /etc/nginx/conf.d/default.conf
```

Убедитесь, что nginx успешно стартовал с новой конфигурацией.

**Задание 7.3.** Комбинированное использование
Создайте ConfigMap app-config с двумя
ключами: database.url (значение: jdbc:postgresql://localhost:5432/mydb) и log-level (значение: DEBUG).

Создайте Pod, который:
Использует значение log-level как переменную окружения LOG_LEVEL.
Монтирует ключ database.url в файл /app/config/db.properties.

*Проверочные команды:*
Проверьте переменную окружения внутри Pod.
Проверьте содержимое файла /app/config/db.properties.

**Задание 7.4.** Диагностика и проблемы

Создайте ConfigMap с несуществующим ключом и попробуйте использовать его в Pod.
Изучите сообщения об ошибках.
Создайте Pod, который монтирует ConfigMap, а затем удалите саму ConfigMap. Что произойдет с Pod?

*Проверочные команды:*
```
kubectl describe pod <problem-pod-name>
kubectl get events --field-selector involvedObject.name=<problem-pod-name>
```

**Задание 7.5.** Веб-приложение с динамическим контентом

Создайте веб-приложение, которое отображает информацию из ConfigMap, и обеспечьте к нему внешний доступ.

Создайте ConfigMap web-content со следующими данными:
- page.title: "Мое приложение"
- welcome.text: "Добро пожаловать в демонстрационное приложение!"
- footer.info: "Версия 1.0 | ConfigMap Demo"

Создайте Deployment web-demo с:
- 2 репликами
- образом nginx:alpine
- монтированием ConfigMap в директорию /usr/share/nginx/html
- использованием всех ключей из ConfigMap как отдельных файлов

Создайте Service web-demo-service типа NodePort на порту 30085

Добавьте метки app: web-demo ко всем ресурсам

*Проверочные команды:*

Проверка создания ресурсов
```
kubectl get configmap web-content -o yaml
kubectl get pods -l app=web-demo --show-labels
kubectl get service web-demo-service
```

Проверка содержимого в Pod
```
kubectl exec <pod-name> -- ls -la /usr/share/nginx/html/
kubectl exec <pod-name> -- cat /usr/share/nginx/html/page.title
```

Тестирование доступа
```
curl http://<node-ip>:30085
```
Или через port-forward
```
kubectl port-forward service/web-demo-service 8080:80
```

## 4. Контрольные вопросы

1. Какие три основных способа использования ConfigMap в Pod вы знаете?
2. Что произойдет, если смонтировать ConfigMap как Volume и затем изменить данные в ConfigMap? Обновится ли конфигурация в работающем Pod?
3. В чем разница между монтированием ConfigMap с секцией items и без нее?
4. Как можно передать все данные из ConfigMap в Pod в виде переменных окружения одной инструкцией?
5. Что произойдет с Pod, если он ссылается на несуществующую ConfigMap?
6. Какие команды вы бы использовали для диагностики проблем с ConfigMap?
7. Может ли один Pod использовать несколько разных ConfigMap? Если да, то как?
8. Как переименовать файл при монтировании ключа из ConfigMap?